

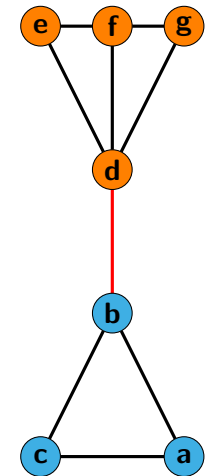
Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

- ▶ we formalize the **common mathematical language** used to analyze **graph-structured data**
- ▶ we introduce a graph-theoretic approach for unsupervised **community detection in networks**
- ▶ we show how it can be addressed based on the **eigenvalue spectrum of the graph Laplacians**

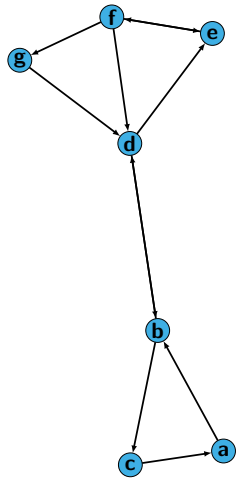


Notes:

- **Lecture L19: Graphs, Laplacians, and Minimum Cuts** 14.07.2025
- **Educational objective:** We show how we can mathematically represent graphs and networks. We introduce basic graph-theoretic concepts and show how we can use the graph Laplacian to detect communities in networks.
 - Graphs, networks, adjacency matrix
 - Paths and connected components
 - Communities, Minimal Cuts and Connectivity
 - Graph Laplacians and Spectral Clustering
- **Exercise sheet 10:** Minimal Cuts in Graphs due 21.07.2025
- **Handout version:** 2025/06/24 15:03:55

Notes:

What is a graph?



graph or network

A **graph or network** is a tuple $G = (V, E)$ where

► V is a set of vertices or **nodes**

► $E \subseteq V \times V$ is a set of edges or **links**

$V \times V$ denotes the Cartesian product of the node set, i.e. the set of all possible links $(i, j) \in V \times V$.

► we say: link (i, j) points **from** node i **to** j

► if not defined otherwise $n := |V|$ $m := |E|$

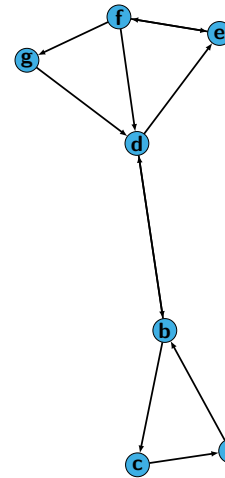
► **multigraphs** can have multiple links between the same nodes, i.e. E is **multiset**

example network

$$V = \{a, b, c, d, e, f, g\}$$

$$E = \{(a, b), (b, c), (b, d), (c, a), (d, b), (d, e), (e, f), (f, d), (f, e), (f, g), (g, d)\}$$

Adjacency matrix



► **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ of graph $G = (V, E)$ is a matrix with

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{else} \end{cases}$$

where A_{ij} refers to **row** i and **column** j

adjacency matrix of directed graph

$$A = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

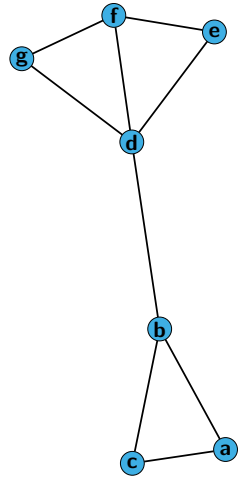
► for directed networks (no self-loops)

$$|E| = m = \sum_{i, j \in V} A_{ij}$$

Notes:

Notes:

Undirected networks



- graph is **undirected** iff

$$(i, j) \in E \Leftrightarrow (j, i) \in E$$

and **directed** otherwise

- adjacency matrices of undirected graphs are **symmetric**, i.e. $A_{ij} = A_{ji} \forall i, j \in V$

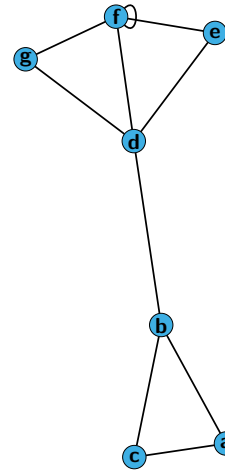
adjacency matrix of undirected graph

$$\mathbf{A} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

- for undirected graphs (no self-loops)

$$|E| = m = \frac{1}{2} \sum_{i,j \in V} A_{ij}$$

Self-loops



- links (i, i) are called **self-loops**

- captured in the **diagonal entries** of adjacency matrix $\mathbf{A}$

- two different representations of self-loops:

1. $(i, i) \in E \implies A_{ii} = 1$

2. $(i, i) \in E \implies A_{ii} = 2$

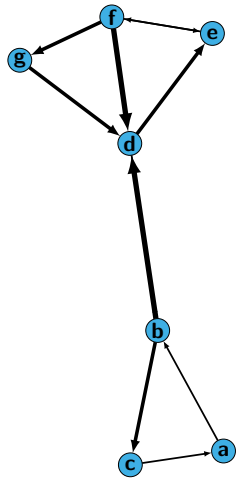
adjacency matrix of graph with self-loop

$$\mathbf{A} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & \mathbf{2} & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Notes:

Notes:

Weighted graphs



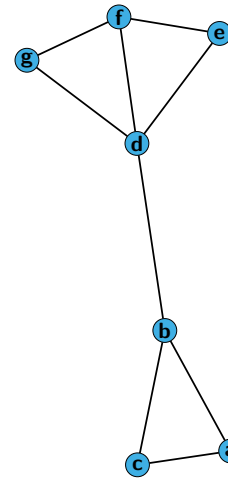
- ▶ in **weighted** graphs links have **numerical attributes** $w : E \rightarrow \mathbb{R}$ that capture strength, frequency, capacity, etc. of links
- ▶ weighted graphs have a **real-valued adjacency matrix** $\mathbf{A} \in \mathbb{R}^{n \times n}$ with

$$A_{ij} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \\ 0 & \text{else} \end{cases}$$

adjacency matrix of weighted network

$$\mathbf{A} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 & 1 & 0 & 2 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Node degrees



undirected graphs

- ▶ **degree** $d(i) = d_i$ of node i is defined as

$$d_i := |\{j \in V : (i,j) \in E\}|$$

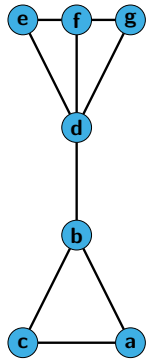
degree matrix of undirected graph

$$\mathbf{D} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 \end{bmatrix} \end{matrix}$$

Notes:

Notes:

Walks, paths and cycles



- ▶ sequence $(p_0, p_1, \dots, p_l)$ of nodes $p_i \in V$ is a **walk** from p_0 to p_l iff

$$(p_i, p_{i+1}) \in E \text{ for } i = 0, \dots, l-1$$

- ▶ walk $(p_0, p_1, \dots, p_l)$ is a **(simple) path** iff $p_i \neq p_j$ for $0 \leq i, j \leq l$ and $i \neq j$

- ▶ walk $(p_0, p_1, \dots, p_l)$ is a **cycle** iff
 1. $p_0 = p_l$
 2. $p_i \neq p_j$ for $0 < i, j < l$ and $i \neq j$

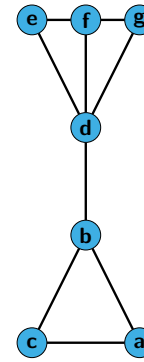
- ▶ **length of path, walk, or cycle** is defined as $\text{len}(p_0, \dots, p_l) := l$

- ▶ two paths are **edge-independent** iff they do not have an edge in common

example

- ▶ (a, b, a, b, d) is a walk
- ▶ (a, b, c, a) is a cycle
- ▶ (a, b, d, g) is path of length three from a to g
- ▶ (a, b) and (a, c, b) are edge-independent

Topological distance



- ▶ **distance** $\text{dist}(v, w)$ between nodes v and w is the minimum length of any path between v and w

- ▶ $\text{dist}(v, w) := \infty \Leftrightarrow \nexists$ path from v to w

- ▶ path $(p_0, \dots, p_l)$ is **shortest path** iff $\text{len}(p_0, \dots, p_l) = \text{dist}(p_0, p_l)$

- ▶ **diameter** $\text{diam}(G)$ of network $G = (V, E)$ is length of the longest shortest path

$$\text{diam}(G) := \max_{v, w \in V} \text{dist}(v, w)$$

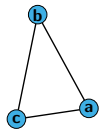
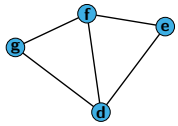
example

- ▶ $\text{dist}(a, d) = 2$
- ▶ shortest path: (a, b, d)
- ▶ $\text{diam}(G) = 3$

Notes:

Notes:

Connected components

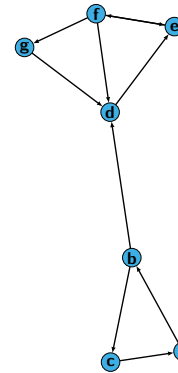


- ▶ undirected graph $G = (V, E)$ is **connected** if $\text{dist}(v, w) < \infty$ for all $v, w \in V$
- ▶ **connected components** of $G = (V, E)$ are maximally connected subgraphs $G' = (V', E')$ with $V' \subseteq V$ and $E' \subseteq E$
- ▶ size of connected component $G' = (V', E')$ is $|V'|$

example

- ▶ connected component $\{a, b, c\}$
- ▶ largest connected component $\{d, e, f, g\}$

Connected components in directed graphs



- ▶ we distinguish between **strongly** and **weakly connected** directed graphs
- ▶ directed network is **weakly connected** iff corresponding undirected graph is connected
- ▶ directed graph $G = (V, E)$ is **strongly connected** iff $\text{dist}(v, w) < \infty \quad \forall v, w \in V$
- ▶ **strongly connected components** of $G = (V, E)$ are maximal strongly connected subgraphs $G' = (V', E')$ with $V' \subseteq V$ and $E' \subseteq E$

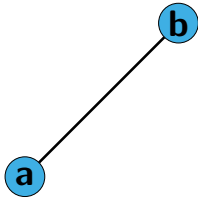
example

- ▶ weakly but not strongly connected
- ▶ strongly connected components $\{a, b, c\}, \{d, e, f, g\}$

Notes:

Notes:

Powers of adjacency matrices



- consider a binary adjacency matrix of a network $G = (V, E)$ with $V = \{a, b\}$ and

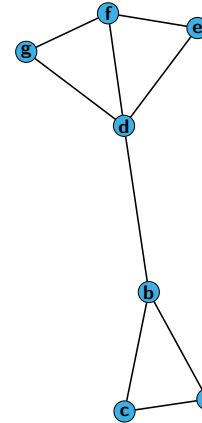
$$\mathbf{A} =: \mathbf{A}^1 = \begin{bmatrix} \delta_{aa} & \delta_{ab} \\ \delta_{ba} & \delta_{bb} \end{bmatrix}$$

with $\delta_{ij} = 1$ if $(i, j) \in E$ and 0 otherwise

- let us **multiply the adjacency matrix** of G **with itself**

$$\mathbf{A}^2 = \begin{bmatrix} \delta_{aa} & \delta_{ab} \\ \delta_{ba} & \delta_{bb} \end{bmatrix} \cdot \begin{bmatrix} \delta_{aa} & \delta_{ab} \\ \delta_{ba} & \delta_{bb} \end{bmatrix} = \begin{bmatrix} \delta_{aa}\delta_{aa} + \delta_{ab}\delta_{ba} & \delta_{aa}\delta_{ab} + \delta_{ab}\delta_{bb} \\ \delta_{ba}\delta_{aa} + \delta_{bb}\delta_{ba} & \delta_{ba}\delta_{ab} + \delta_{bb}\delta_{bb} \end{bmatrix}$$

Powers of adjacency matrices



$$\mathbf{A}^3 = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & 4 & 3 & 1 & 1 & 1 & 1 \\ 4 & 2 & 4 & 6 & 1 & 2 & 1 \\ 3 & 4 & 2 & 1 & 1 & 1 & 1 \\ 1 & 6 & 1 & 4 & 6 & 6 & 6 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \\ 1 & 2 & 1 & 6 & 5 & 4 & 5 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \end{bmatrix} \end{matrix}$$

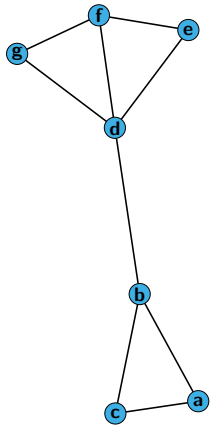
interpretation of $\mathbf{A}^k$

entries A_{ij}^k of k -th power of adjacency matrix count **different walks of exactly length k** between node i and node j

Notes:

Notes:

Adjacency matrices and linear algebra



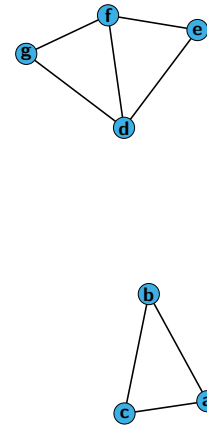
$$\mathbf{A}^3 = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & 4 & 3 & 1 & 1 & 1 & 1 \\ 4 & 2 & 4 & 6 & 1 & 2 & 1 \\ 3 & 4 & 2 & 1 & 1 & 1 & 1 \\ 1 & 6 & 1 & 4 & 6 & 6 & 6 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \\ 1 & 2 & 1 & 6 & 5 & 4 & 5 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \end{bmatrix} \end{matrix}$$

example graph

entries of third power $\mathbf{A}^3$ of adjacency matrix tell us that

1. graph is connected
2. diameter is at most three

Connected components



$$\mathbf{A}^3 = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & 3 & 3 & 0 & 0 & 0 & 0 \\ 3 & 2 & 3 & 0 & 0 & 0 & 0 \\ 3 & 3 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 5 & 5 & 5 \\ 0 & 0 & 0 & 5 & 2 & 5 & 2 \\ 0 & 0 & 0 & 5 & 5 & 4 & 5 \\ 0 & 0 & 0 & 5 & 2 & 5 & 2 \end{bmatrix} \end{matrix}$$

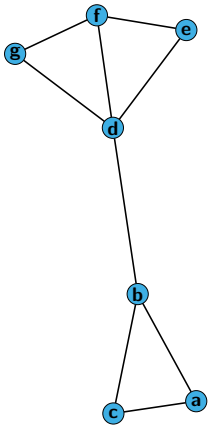
example network

for sufficiently large k and suitably ordered matrix rows and columns, connected components show as **blocks** in $\mathbf{A}^k$

Notes:

Notes:

What can you see here?



$$\mathbf{A}^3 = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & 4 & 3 & 1 & 1 & 1 & 1 \\ 4 & 2 & 4 & 6 & 1 & 2 & 1 \\ 3 & 4 & 2 & 1 & 1 & 1 & 1 \\ 1 & 6 & 1 & 4 & 6 & 6 & 6 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \\ 1 & 2 & 1 & 6 & 5 & 4 & 5 \\ 1 & 1 & 1 & 6 & 2 & 5 & 2 \end{bmatrix} \end{matrix}$$

example network

- ▶ blocks of larger values in $\mathbf{A}^3$ are due to **clusters or communities** in the network
- ▶ block structure depends on ordering of rows/columns
- ▶ basis for spectral clustering algorithms

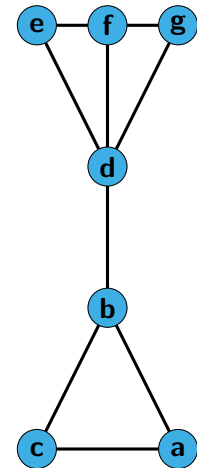
From components to communities ...

- ▶ connected component = subset $C \subseteq V$ of nodes **exclusively connected** to other nodes in C
- ▶ yields (trivial) definition of **clusters** in a graph
- ▶ we need a “soft” definition that **allow (few) links between clusters**

communities in graphs

a subset $C \subset V$ of nodes is called **community** if nodes in C are “more strongly connected” to other nodes in C than to nodes not in C .

- ▶ generalizes definition of **clusters in Euclidean space** to graph-shaped data



Notes:

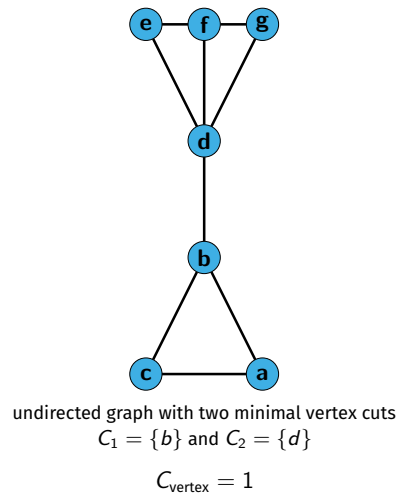
Notes:

Vertex cuts and vertex connectivity

- ▶ consider undirected graph $G = (V, E)$
- ▶ **vertex cut** is subset $C \subseteq V$ such that $G = (V - C, E)$ is disconnected
- ▶ number of nodes $|C|$ in vertex cut C is called **vertex cut size**
- ▶ **vertex connectivity** C_{vertex} of a graph G is minimal vertex cut size, i.e.

$$C_{\text{vertex}} := \min\{|C| : C \subseteq V \text{ vertex cut for } G\}$$

- ▶ for graph with n nodes vertex connectivity $C_{\text{vertex}} \in \{0, 1, \dots, n - 1\}$
- ▶ **vertex connectivity generalizes binary definition of connectedness**



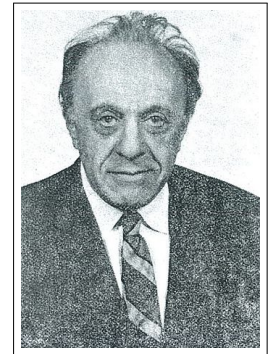
Menger's theorem

- ▶ vertex connectivity of a graph is related to number of edge-independent paths

Menger's theorem

in a finite graph, the **minimum size of a vertex cut** is equal to the **maximum number of edge-independent paths** between any pair of vertices.

- ▶ small vertex connectivity indicates **“bottleneck” in topology**, i.e. nodes connected by small number of edge-independent paths
- ▶ formalizes definition of communities as **groups of “well-connected” nodes**



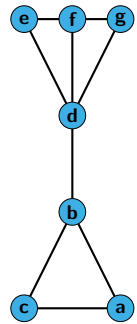
Karl Menger
1902 – 1985

image credit: Shimer College, Public domain

Notes:

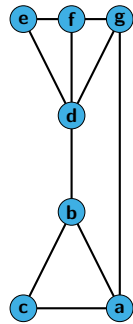
Notes:

Example: vertex connectivity and paths



$$C_{\text{vertex}} = 1$$

nodes a and g connected by **single edge-independent path**
 $\{(a, b, d, g)\}$



$$C_{\text{vertex}} = 2$$

nodes a and g connected by **two edge-independent paths** $\{(a, b, d, g), (a, g)\}$

Edge cuts and normalized cut size

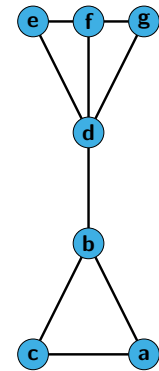
- **edge cut** is a partition of nodes into two disjoint subsets $C_1, C_2 \subset V$ with $C_1 \cup C_2 = V$ and $C_1 \cap C_2 = \emptyset$
- we call number of edges crossing C_1 and C_2 **size of edge cut**, i.e.

$$s(C_1, C_2) = |\{(v, w) \in E : v \in C_1, w \in C_2\}|$$

- **normalized cut size** is given as

$$N_s(C_1, C_2) = s(C_1, C_2) \left(\frac{1}{\sum_{v \in C_1} d_v} + \frac{1}{\sum_{v \in C_2} d_v} \right)$$

- computation of edge cuts with minimal normalized size is **NP-hard** → Shi and Malik, 2000



network with minimal edge cut
 $C_1 = \{a, b, c\}, C_2 = \{d, e, f, g\}$
 $N_s(C_1, C_2) = 1 \cdot \left(\frac{1}{7} + \frac{1}{11} \right) \approx 0.234$

Notes:

Notes:

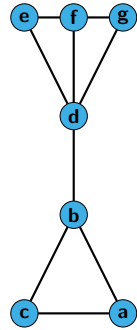
The Laplacian Matrix

- for **undirected graph** consider **Laplacian matrix**

$$\mathcal{L} := \mathbf{D} - \mathbf{A}$$

where $\mathbf{D}$ is degree-diagonal matrix, i.e.
 $D_{ii} := d_i$

- rows and columns of graph Laplacian sum to zero



$$\mathcal{L} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & -1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 & 0 & 0 \\ -1 & -1 & 2 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 4 & -1 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & -1 & 3 & -1 \\ 0 & 0 & 0 & -1 & 0 & -1 & 2 \end{bmatrix} \end{matrix}$$

Algebraic Connectivity

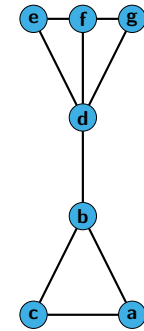
- consider eigenvalues
 $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ of Laplacian $\mathcal{L}$ in **ascending order**
- $(1, \dots, 1) \cdot \mathcal{L} = \vec{0}$ so $\vec{1}$ is eigenvector for **smallest eigenvalue** $\lambda_1 = 0$

algebraic connectivity

we call second-smallest eigenvalue λ_2 of the graph Laplacian **algebraic connectivity**

- algebraic connectivity is **lower bound for vertex connectivity**

$$\lambda_2 \leq C_{\text{vertex}}$$



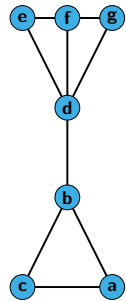
$$\mathcal{L} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 2 & -1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 & 0 & 0 \\ -1 & -1 & 2 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 4 & -1 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & -1 & 3 & -1 \\ 0 & 0 & 0 & -1 & 0 & -1 & 2 \end{bmatrix} \end{matrix}$$

$$\lambda_2 = 0.398$$

Notes:

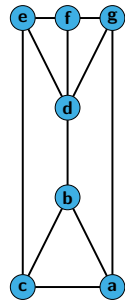
Notes:

Example: algebraic connectivity



$$C_{\text{vertex}} = 1$$

$$\lambda_2 \approx 0.398 < 1$$



$$C_{\text{vertex}} = 3$$

$$\lambda_2 \approx 1.596 < 3$$

algebraic connectivity can be used to assess **how “well-connected”** a graph is, i.e. **whether topology contains “bottlenecks”**

Fiedler vector

- consider sequence of **eigenvectors**

$$\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$$

corresponding to eigenvalues $\lambda_1 \leq \dots \leq \lambda_n$ of $\mathcal{L}$ in ascending order

- due to $(1, \dots, 1) \cdot \mathcal{L} = \vec{0}$ we have eigenvector $\vec{v}_1 = (1, \dots, 1)$ corresponding to **smallest eigenvalue** $\lambda_1 = 0$

Fiedler vector

- eigenvector $\vec{v}_2$ corresponding to λ_2 is called **Fiedler vector**
→ M Fiedler, 1973
- entries can be used to **find edge cuts with minimal normalized size**



Miroslav Fiedler
1926 – 2015

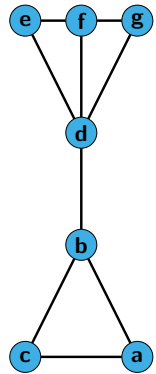
image credit: <http://www.cs.cas.cz/fiedler/>

Notes:

Notes:

Example: Fiedler vector

1/2

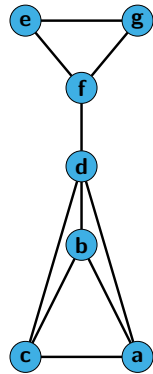


$$C_{\text{vertex}} = 1, \lambda_2 \approx 0.398$$

minimal edge cut $\{C_1 = \{a, b, c\}, C_2 = \{d, e, f, g\}\}$

$$N_s(C_1, C_2) \approx 0.234$$

$$\vec{v}_2 = (-0.49, -0.3, -0.49, 0.21, 0.36, 0.36, 0.36)$$



$$C_{\text{vertex}} = 1, \lambda_2 \approx 0.398$$

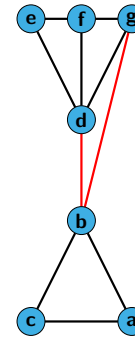
minimal edge cut $\{C_1 = \{a, b, c, d\}, C_2 = \{e, f, g\}\}$

$$s(C_1, C_2) = 0.22$$

$$\vec{v}_2 = (-0.36, -0.36, -0.36, -0.21, 0.49, 0.3, 0.49)$$

Example: Fiedler vector

2/2

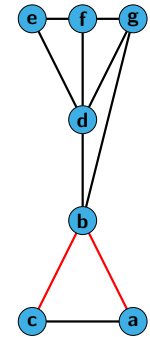


edge cut $\{C_1 = \{a, b, c\}, C_2 = \{d, e, f, g\}\}$

$$N_s(C_1, C_2) \approx 0.417$$

$$C_{\text{vertex}} = 2, \lambda_2 \approx 0.64$$

$$\vec{v}_2 = (0.52, 0.19, 0.52, -0.24, -0.45, -0.37, -0.18)$$



edge cut $\{C_1 = \{a, c\}, C_2 = \{b, d, e, f, g\}\}$

$$N_s(C_1, C_2) \approx 0.625$$

Notes:

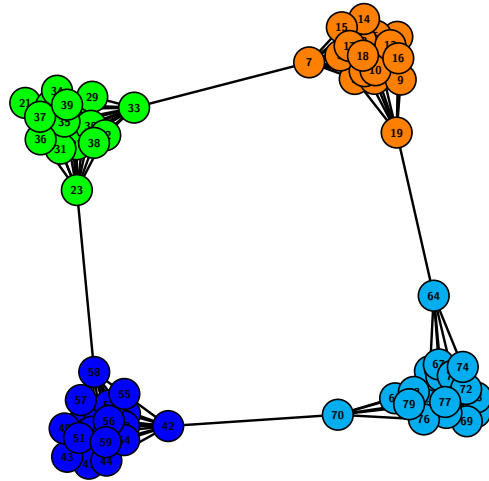
Notes:

Spectral clustering

- ▶ we can use Fiedler vector to **bisect vertices** in a graph
- ▶ bisection is based on **edge cut with minimal normalized size**

spectral clustering algorithm

- ▶ Q: what if a graph contains **more than two clusters**?
- ▶ A: we can **recursively apply bisection** to vertex sets in edge cut
- ▶ Q: when do we **stop recursion**?
- ▶ A: we can use **threshold for algebraic connectivity**



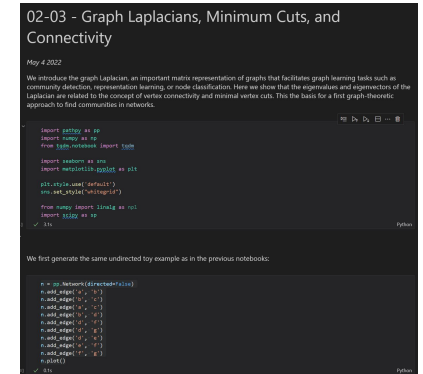
Practice session

- ▶ we explore **minimum cuts and vertex connectivity**
- ▶ we compute **graph Laplacians** and **algebraic connectivity**
- ▶ we study **minimum edge cuts** and compute the **Fiedler vector**
- ▶ we use the Laplacian matrix for **spectral community detection**

practice session

see notebooks 19-01 – 19-02 in [gitlab repository](https://gitlab2.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids/) at

→ https://gitlab2.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids/

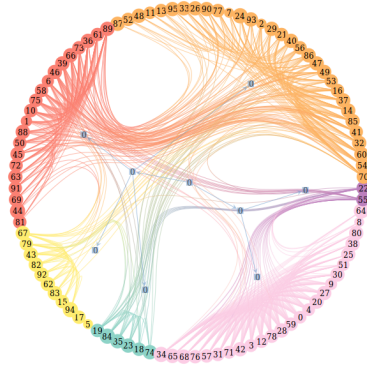


Notes:

Notes:

In summary ...

- ▶ we revisited fundamental **graph-theoretic concepts**
- ▶ we generalized the binary definition of connectedness using **vertex connectivity**
- ▶ we introduced the **graph Laplacian** and its application in **spectral clustering**



communities detected in social network of students at MIT

Self-study questions

1. Give an example for a walk and a path in a network.
2. What is represented by the entries of the k -th power of an adjacency matrix?
3. Explain how we can use adjacency matrix powers $\sum_{k=1}^l \mathbf{A}^k$ to compute the diameter of a network.
4. How is the definition of a community related to edge cuts?
5. Give an example for a network where the sizes of both the minimal vertex and edge cut are $k = 5$.
6. Give an example for a network with two edge cuts with size two and different normalized cut sizes.
7. What is the multiplicity of zero in the eigenvalue sequence of the Laplacian for a graph with three connected components?
8. What is the computational complexity of calculating the eigenvalues and eigenvectors of a Laplacian matrix?

Notes:

Notes:

References

reading list

- ▶ M Fiedler: **Algebraic Connectivity of Graphs**, Czechoslovak Mathematical Journal 23, 1973
- ▶ J Shi and J Malik: **Normalized Cuts and Image Segmentation**, IEEE Transactions on Patterns Analysis and Machine Intelligence, 2000
- ▶ M Schaub, JC Delvenne, M Rosvall, R Lambiotte: **The many facets of community detection in complex networks**, Applied Network Science, 2017
- ▶ M Belkin, P Niyogi: **Laplacian Eigenmaps for Dimensionality Reduction and Data Representation**, Neural Computation, 2003 → section 4

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 22, NO. 6, AUGUST 2000

Normalized Cuts and Image Segmentation

Jianbo Shi and Jitendra Malik, Member, IEEE

Abstract—This paper presents a novel approach for solving the perceptual grouping problem in vision. Rather than focusing on local features and their continuities in the image data, our approach aims at reducing the global impedance of an image. We treat image segmentation as a global optimization problem where segments are treated as pixels. The normalized cut criterion characterizes the total impedance between the different pixels as well as the total impedance within the groups. We show that an efficient computational technique based on a generalized eigenvalue problem can be used to optimize the criterion. We have applied this approach to segmenting static images, as well as motion sequences, and found the results to be very encouraging.

Index Terms—Grouping, image segmentation, graph partitioning.

1 INTRODUCTION

NINETY-SEVEN years ago, Wertheimer [24] pointed out the importance of perceptual grouping and organization in vision and listed several key factors, such as similarity, proximity, and good continuation, which lead to visual grouping. However, even to this day, many of the computational issues of perceptual grouping have remained unresolved. In this paper, we present a general framework for this problem, focusing specifically on the case of image segmentation.

Since there are many possible partitions of the domain of an image into subsets, how do we pick the “right” one? There are two aspects to be considered here. The first is that there may not be a single correct answer. A Bayesian view is appropriate—there are several possible interpretations in the context of prior world knowledge. The difficulty, of course, is in specifying the prior world knowledge. Some of it is low level, such as coherence of brightness, color, texture, or motion, but equally important is mid- or high-level knowledge about organization of shapes or object models. The second aspect is that the partitioning is inherently hierarchical. Therefore, it is more appropriate to think of returning a tree structure corresponding to a hierarchical partition instead of a single “flat” partition.

This suggests that image segmentation based on low-level cues (edges) should not aim to produce a complete final “correct” segmentation. The objective should instead be to use the low-level evidence of brightness, color, texture, or motion attributes in conjunction with hierarchical partitions. Mid- and high-level knowledge can be used to either confirm these groups or select some for further attention. This attention could result in further segmentation or grouping. The key point is that image partitioning is

to be done from the big picture downward, rather like a painter first marking out the major areas and then filling in the details.

Four decades on the related problems of clustering, grouping, and image segmentation is long. The clustering community [23] has defined an algorithmic and device algorithms in image segmentation, we have experienced merge-and-split algorithms. The hierarchical divide-and-conquer approach that we advocate produces a tree, the desktop. While most of these ideas go back to the 1970s (and earlier), the 1980s brought to the use of Markov Random Fields [10] and variational formulations [15], [25], [14]. The MRF and variational formulations also exposed two basic questions:

1. What is the criterion that one wants to optimize?
2. Is there an efficient algorithm for carrying out the optimization?

Many an attractive criterion has been devised by the inability to find an effective algorithm to find it. Minimum-cut or gradient descent type approaches fail to find global optima for these high-dimensional, nonlinear problems.

Our approach is most related to the graph theoretic formulation of grouping. The set of pixels in an arbitrary feature space are represented as a weighted undirected graph $G = (V, E)$, where the nodes of the graph are the pixels in the feature space, and an edge is formed between every pair of nodes. The weight on each edge, w_{ij} , is a function of the similarity between nodes i and j .

In grouping, we seek to partition the set of vertices into disjoint sets $V_1, V_2, \dots, V_k$, where for some reason the similarity among the vertices in a set V_i is high and across different sets V_i, V_j is low.

To partition a graph, we need to also ask the following questions:

1. What is the precise criterion for a good partition?
2. How can such a partition be computed efficiently?

In the image segmentation and data clustering community, there has been much previous work using variations of the minimal spanning tree or limited neighborhood approaches. Although these use efficient computational

• J. Shi is with the Robotics Institute, Carnegie Mellon University, 5000
Pittsburgh, PA 15261, E-mail: jshi@cmu.edu.

• J. Malik is with the Computer Engineering Department, University of California at Berkeley, Berkeley, CA 94720, E-mail: malik@eecs.berkeley.edu.

Manuscript received 2 Jan. 1998; revised 10 Dec. 1998.
Revised manuscript received 10 Jan. 1999.

This paper is an extended version of the article, plus and used in the proceedings of the IEEE Conference on Computer Vision and Pattern Recognition and reference IEEECS Log Number 197224.

© 2000 IEEE

Notes: